Hash maps or hash tables

- Store (key,value) pairs using an array of size m
- Insert, search and delete operations
- Compute an index from the key (modulo m) using a hash function
- When two keys are mapped to the same index there is a collision
- Two main approaches to handle collisions:
  - with separate chaining (öppen hashtabell)
  - with open addressing (sluten hashtabell)
- With n pairs, $\alpha = \frac{n}{m}$ is the load factor
- Separate chaining uses linked lists for the pairs
- Open addressing stores the pairs in the array

Separate chaining

- The table is an array of linked lists
- Compared with only one list, this will likely be m times faster
- Some alternatives:
  - Always insert at the beginning (no search needed if you know it is a new pair)
  - Keep each list sorted
  - Move frequently used pairs to the beginning of the list
- Advantage: simple to implement
- Disadvantage: less simple to allocate memory for the list nodes efficiently (allocation and freeing/garbage collecting nodes takes time)
- Usually a good choice
- If α gets too big, the operations will be slower but still work
- Resize array if needed

Open addressing

- Invented by Gene Amdahl (known for Amdahl's law)
- $\alpha < 1$ (but see below — significantly less than one is better)
- Three ways to handle collisions
  - Linear probing
  - Quadratic probing
  - Double hashing probing

Linear probing

- An array element either contains a pair or a value "empty" (e.g. null)
- Sometimes it is simpler to have one array for keys and another for values, with the key and value of a pair stored at index i in the two arrays
- First compute an index $i \leftarrow f(key) \mod m$
- If $a[i]$ is not empty, set $i \leftarrow (i+1) \mod m$ and check again
- Otherwise insert new pair at i
- Similar for search
- Quiz: can we do the same for delete plus storing "empty" in the array?

Linear probing: delete

- Answer: no, since we then might not find some of the keys.
- Empty hash table:

empty	empty	empty	empty	empty	empty	empty

- Insert two pairs with $f(k_1) \mod m = f(k_2) \mod m$

empty	k_1	k_2	empty	empty	empty	empty

- Delete the pair k_1

empty	empty	k_2	empty	empty	empty	empty

- Search for k_2

empty	empty	k_2	empty	empty	empty	empty

will give up at first probe since it sees "empty"

- What can we do? Quiz: why not store a value "deleted" and skip such when searching?

Linear probing: delete

- Answer: yes, that works but gives other problems
- Empty hash table:

empty	empty	empty	empty	empty	empty	empty

- Insert two pairs with $f(k_1) \mod m = f(k_2) \mod m$

empty	k_1	k_2	empty	empty	empty	empty

- Delete the pair k_1

empty	deleted	k_2	empty	empty	empty	empty

- Search for k_2

empty	deleted	k_2	empty	empty	empty	empty

will skip "deleted" and find k_2

- Quiz: when is this bad?

Storing "deleted"

- Answer: we may store many "deleted" that must be skipped
- But if we insert and come to a "deleted" then we can use that position.
- I use the words position and index for the same place in the array but it is a slight abuse of English.
- If we have too many "deleted" we can clean the hash table to remove them by reinserting everything
- Quiz: instead of storing "deleted", can we not move items "to the left"?

Linear probing: delete with move

- Answer: yes, if we are careful
- Empty hash table:

empty	empty	empty	empty	empty	empty	empty

- Insert two pairs with $f(k_1) \mod m = f(k_2) \mod m$

empty	k_1	k_2	empty	empty	empty	empty

- Delete the pair k_1 and move k_2

empty	k_2	empty	empty	empty	empty	empty

- Search for k_2

empty	k_2	empty	empty	empty	empty	empty

will find k_2

- Quiz: what could go wrong?

Linear probing: delete with bad move

- Answer: moving a key to the left of its originally computed index
- Insert two pairs with $f(k_1) \mod m = f(k_2) \mod m$

empty	k_1	k_2	empty	empty	empty	empty

- Insert a pair with $f(k_0) \mod m = 0$

k_0	k_1	k_2	empty	empty	empty	empty

- Delete k_1 and move k_2

k_0	k_2	empty	empty	empty	empty	empty

- Delete k_0 and move k_2

k_2	empty	empty	empty	empty	empty	empty

will never find k_2

Linear probing: correct delete with move

```
procedure delete(k, h)
begin
    i ← position(k)
    if i < 0 then
        return
    while true do {
        a[i] ← empty
        j ← i
        while true do {
            i ← (i + 1) mod m
            if a[i] = empty then
                return
            k ← h(a[i]) mod m
            if not ((j < k and k ≤ i) or (i < j and ((j < k) or k ≤ i))) then
                a[j] ← a[i]
                break
    }
end
```

A simple model of unsuccessful search in linear probing

- We ignore clustering and instead assume all positions are equally likely to be occupied.
- If the random variable X is the number of probes in an unsuccessful search, what is the expected value of X, $\mathbb{E}[X]$?
- $\mathbb{P}(X \geq k)$ is the probability that the first $k-1$ positions are occupied, and the last is empty.
  - The probability the first probed is occupied is: $\frac{n}{m}$,
  - the two first: $\frac{n}{m} \cdot \frac{n-1}{m-1}$,

- For $k > 1$ we can write:

$$\mathbb{P}(X \geq k) \quad = \quad \frac{n}{m} \cdot \frac{n-1}{m-1} \cdot \ldots \cdot \frac{n-k+2}{m-k+2} \leq \left(\frac{n}{m}\right)^{k-1} = \alpha^{k-1},$$

- with the last expression is valid also for $k = 1$.

A simple formula for $\mathbb{E}[X]$

$$
\begin{aligned}
\mathbb{E}[X] &= \sum_{k=1}^{\infty} k \cdot \mathbb{P}(X = k) \\
&= \sum_{k=1}^{\infty} k \cdot (\mathbb{P}(X \geq k) - \mathbb{P}(X \geq k+1)) \\
&= 1 \cdot \mathbb{P}(X \geq 1) - 1 \cdot \mathbb{P}(X \geq 2) \\
&\quad + 2 \cdot \mathbb{P}(X \geq 2) - 2 \cdot \mathbb{P}(X \geq 3) \\
&\quad + 3 \cdot \mathbb{P}(X \geq 3) - 3 \cdot \mathbb{P}(X \geq 4) \\
&\quad \dots \\
&= \mathbb{P}(X \geq 1) \\
&\quad + \mathbb{P}(X \geq 2) \\
&\quad + \mathbb{P}(X \geq 3) \\
&\quad \dots \\
&= \sum_{k=1}^{\infty} \mathbb{P}(X \geq k) \\
&\leq \sum_{k=1}^{\infty} \alpha^{k-1} \\
&= \sum_{k=0}^{\infty} \alpha^{k} = \tfrac{1}{1-\alpha}, \text{ since } \alpha < 1.
\end{aligned}
$$

Max expected number of probes in an unsuccessful search

α	$\mathbb{E}[X]$
0.2	1.25
0.3	1.43
0.4	1.67
0.5	2.00
0.6	2.50
0.7	3.33
0.8	5.00
0.9	10.00
0.95	20.00
0.98	50.00

TAOCP = The Art of Computer Programming

- Recall this is an optimistic estimation since clustering is ignored
- In reality, long sequences of occupied positions tend to grow longer
- See Knuth TAOCP Volume 3 for a more detailed analysis
- This analysis is sufficient to convince us to avoid large α

Quadratic probing

- The purpose of quadratic probing is to reduce the risk of clustering by adding i^2 instead of only i to the initial hash value.
- The intent is to leave a cluster quickly.
- Below h' is the original hash function

$$h(k, i) \quad = \quad (h'(k) + i^2) \mod m$$

- Clustering is reduced but if two different keys have the same hash value, there can be secondary clustering since the positions probed for these keys will be the same.
- Quiz: can we now know we will find an empty position if there is one?

Quadratic probing

- Answer: no
- Assume $m = 3$ and $h'(k) = 0$
- This is a very bad "hash function" but for illustration only, but during debugging it can be useful
- The sequence of visited positions would initally be: $(0, 1, 1)$ from $(0, (0 + 1^2) \mod 3 = 1, (0 + 2^2) \mod 3 = 1)$
- We number the probings as probe $0, 1, 2$
- We miss position 2 since different probe numbers are mapped to the same position
- Note it did not help that m is a prime number — at least not for now
- Quiz: can we add a constraint to make this work?

Making quadratic probing work better 1(2)

- Answer: yes.
- If m is prime and we also require that $\alpha = n/m < \frac{1}{2}$, it will work.
- Let i and j be the probe numbers made for two different searches or insertions.
- Assume both operations resulted in the same hash value so they start searching at the same positions.
- Initially $i = j = 1$.
- Then they are used to probe and possibly they are incremented.

Making quadratic probing work better 2(2)

- Assume the operation using i inserted something first, somewhere.
- The operation using j will initially use the same positions, i.e. same values as i.
- We want to show that when i and j have *different values* they would not map to the same positions
- That means j does not "return to" a position in the sequence used by i.
- Recall that was our problem before: $(0, 1, 1)$

A lemma

Lemma

If m is prime and $\alpha = \frac{n}{m} < \frac{1}{2}$, and $i \neq j$, then quadratic probing will find an empty position in less than $\frac{m}{2}$ probes

A proof

Proof.

Let $0 \leq i, j < \lceil \frac{m}{2} \rceil$, and $h'(k_1) = h'(k_2)$. Assume incorrectly that two different probe numbers, i and j, are mapped to the same positions.

$$\begin{aligned}
(h'(k_1) + i^2) \mod m &= (h'(k_2) + j^2) \mod m \\
(h'(k_1) + i^2) &\equiv (h'(k_2) + j^2) \mod m \\
i^2 &\equiv j^2 \mod m \\
i^2 - j^2 &\equiv 0 \mod m \\
(i - j)(i + j) &\equiv 0 \mod m
\end{aligned}$$

Since m is prime and $i \neq j$, either $i - j$ or $i + j$ is divisible by m, but since both $i - j$ and $i + j$ are less than m, none of them can be divisible by m. A contradiction. Therefore the first $\lceil \frac{m}{2} \rceil$ probes are to different positions and since $\alpha < \frac{1}{2}$, an empty position will be found. $\square$

Double hashing

- Another alternative is to use an additional hash function:

$$h(k, i) \quad = \quad (h_1(k) + i \cdot h_2(k)) \quad \text{mod } m$$

- Assume two different keys had the same hash value in quadratic probing, i.e., with $h_1(k)$.
- Then it is hoped that the risk that they have the same value also for $h_2(k)$ is much less.
- In practice this removes most clustering.
- By guaranteeing that $h_2(k)$ is relatively prime to m, all positions will be probed. Two simple ways to achieve this are:
  - let $m = 2^n$ and ensure that $h_2(k)$ always is odd, or
  - let m be prime and $0 < h_2(k) < m$.

Summary of hash tables

- No obvious "best" choice
- Luckily both alternatives are easy to implement.
- Best is to make performance measurements before changing anything, obviously.
- Especially the number of cache misses can explain performance differences.
- See EDAG01 for profilers for C (and C++).
- In EDAG01 you will even use a simulator from IBM which explains what happens each clock cycle in a modern CPU (POWER8).

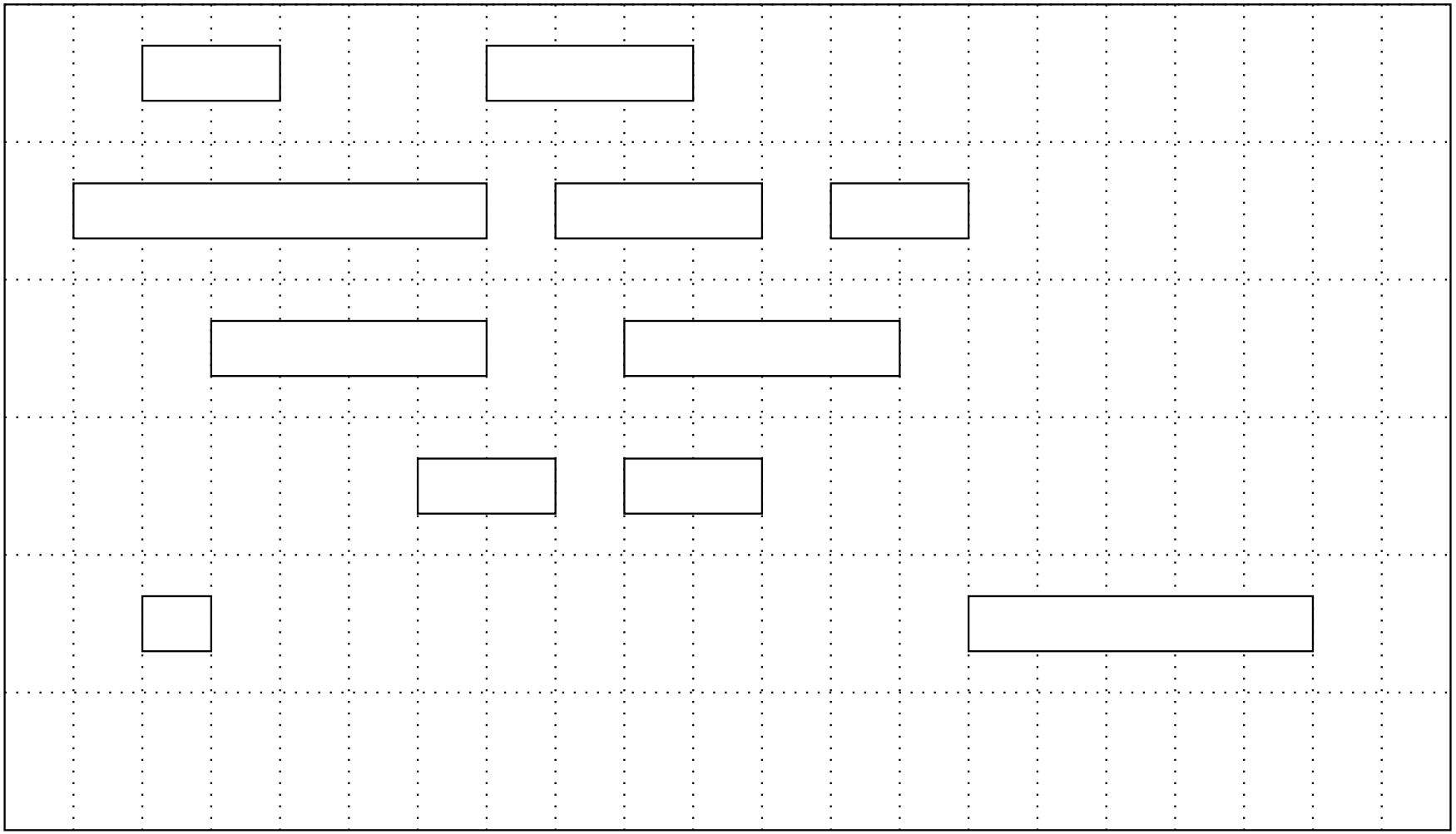
Greedy algorithms / Giriga algoritmer

- It is not trivial to define precisely what makes an algorithm greedy.
- The main idea is to use a simple rule to make decisions without taking "all" information into account.
- The challenge is to find a simple rule which solves a problem optimally
- Two approaches to prove that a greedy algorithm is optimal:
  1. The greedy algorithm "stays ahead" — by proving it is always at least as good as an optimal algorithm
  2. Exchange argument (utbytesargument) — transform the output of an optimal algorithm (without changing its quality) to the output of the greedy algorithm

Interval scheduling / Intervallschemaläggning

- One resource
- A set R of requests, r_i, with a start time $s(i)$ and a finish time $f(i)$
- A set of requests is **compatible** if they do not overlap in time
- The **interval scheduling problem** is to find the largest subset $S \subseteq R$ such that S is compatible
- All requests have equal value and it is the size of S we want to maximize
- A compatible set of maximum size is called an optimal schedule

An example set R



A greedy algorithm for interval scheduling

procedure *schedule*(R)
$S \leftarrow \emptyset$ /* S is a sequence */
while $R \neq null$
    $r \leftarrow$ select a request from R
    remove r from R
    add r to the end of S
    remove all requests in R which overlap with r
return S

- Our problem is to figure out a clever *select* function
- Any suggestions?

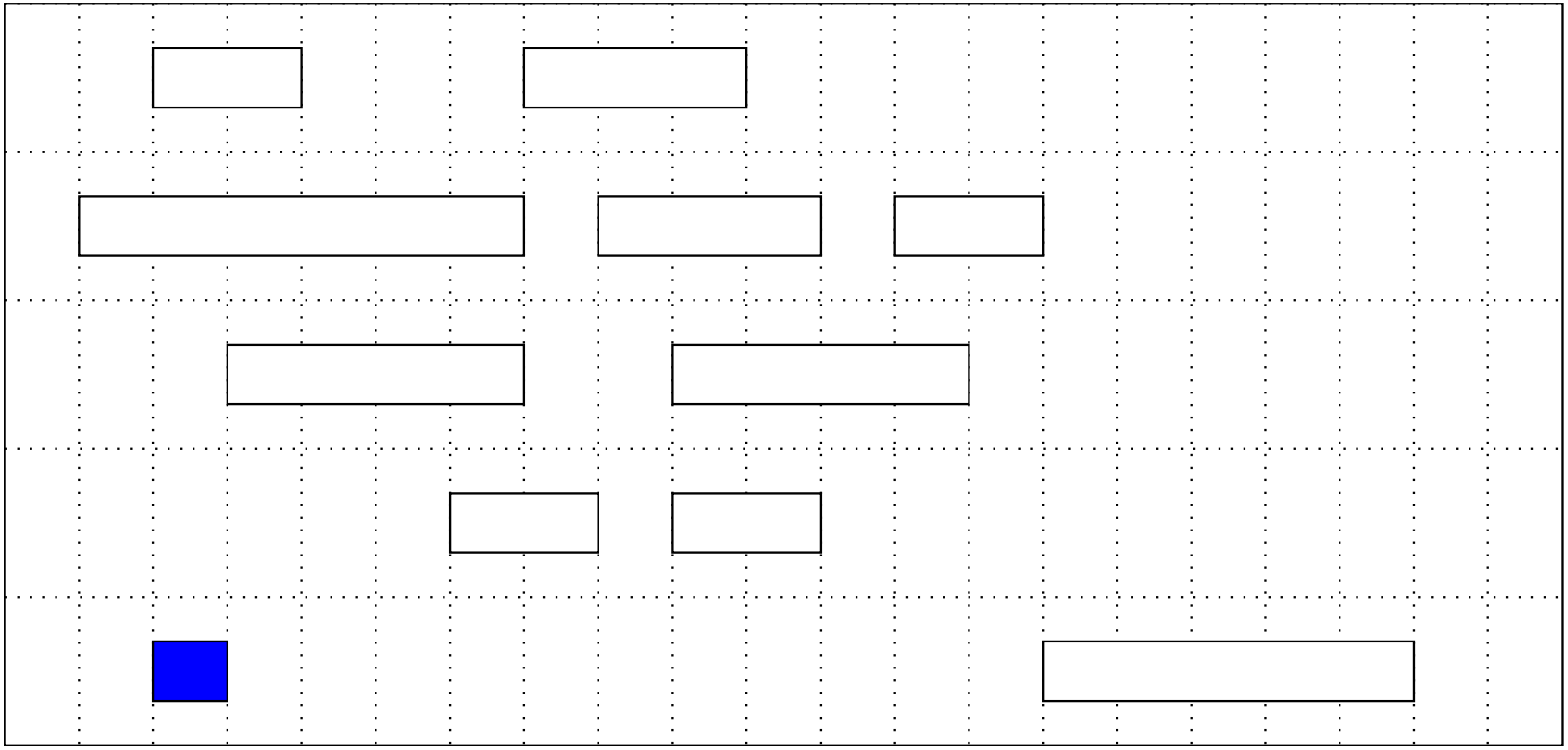
Ideas for the select function

- Take the request with shortest interval
- Take the request which starts first
- Take the request with fewest conflicts

None of these lead to an optimal solution

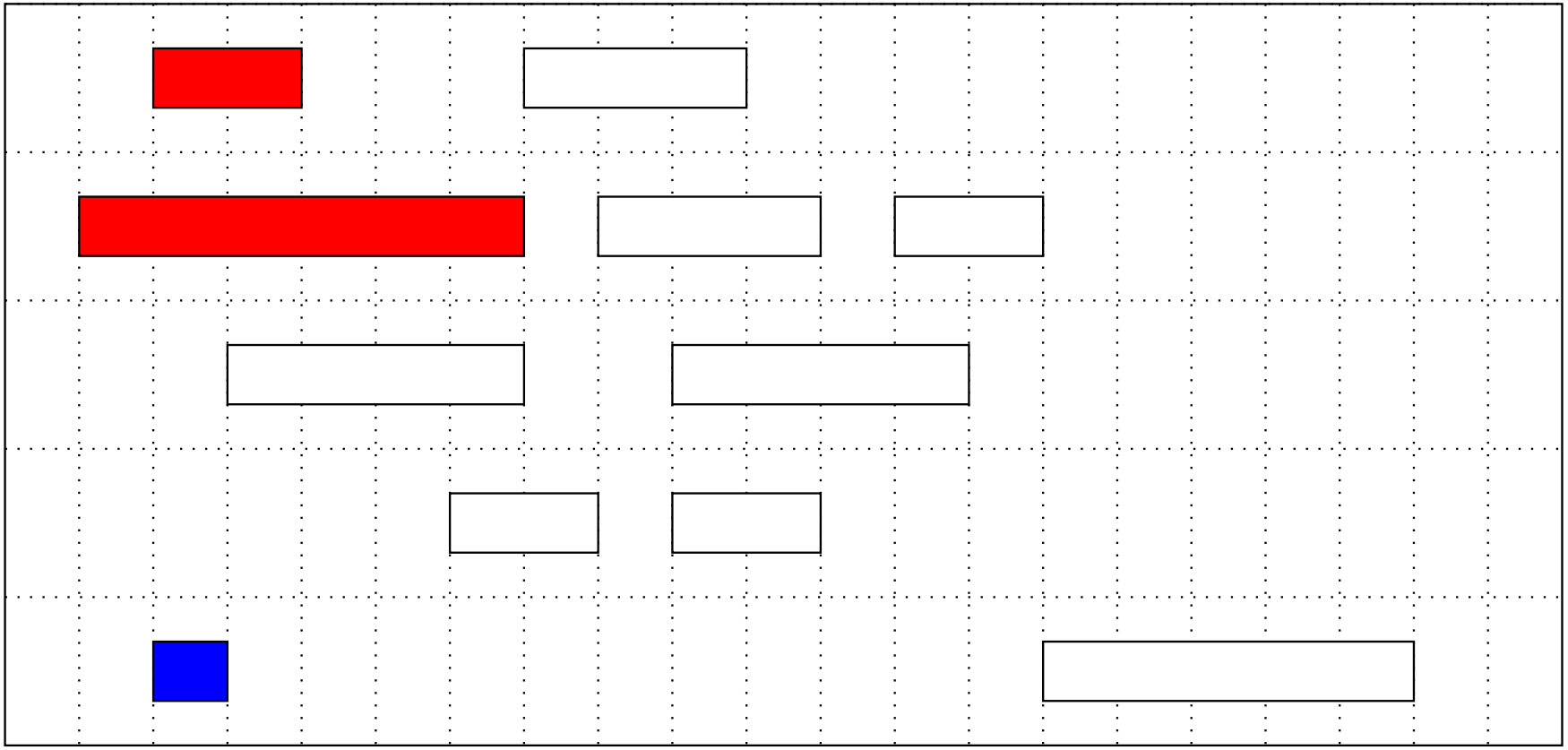
A better select function

- Take the request which finishes earliest
- Is this optimal?
- Select first request:



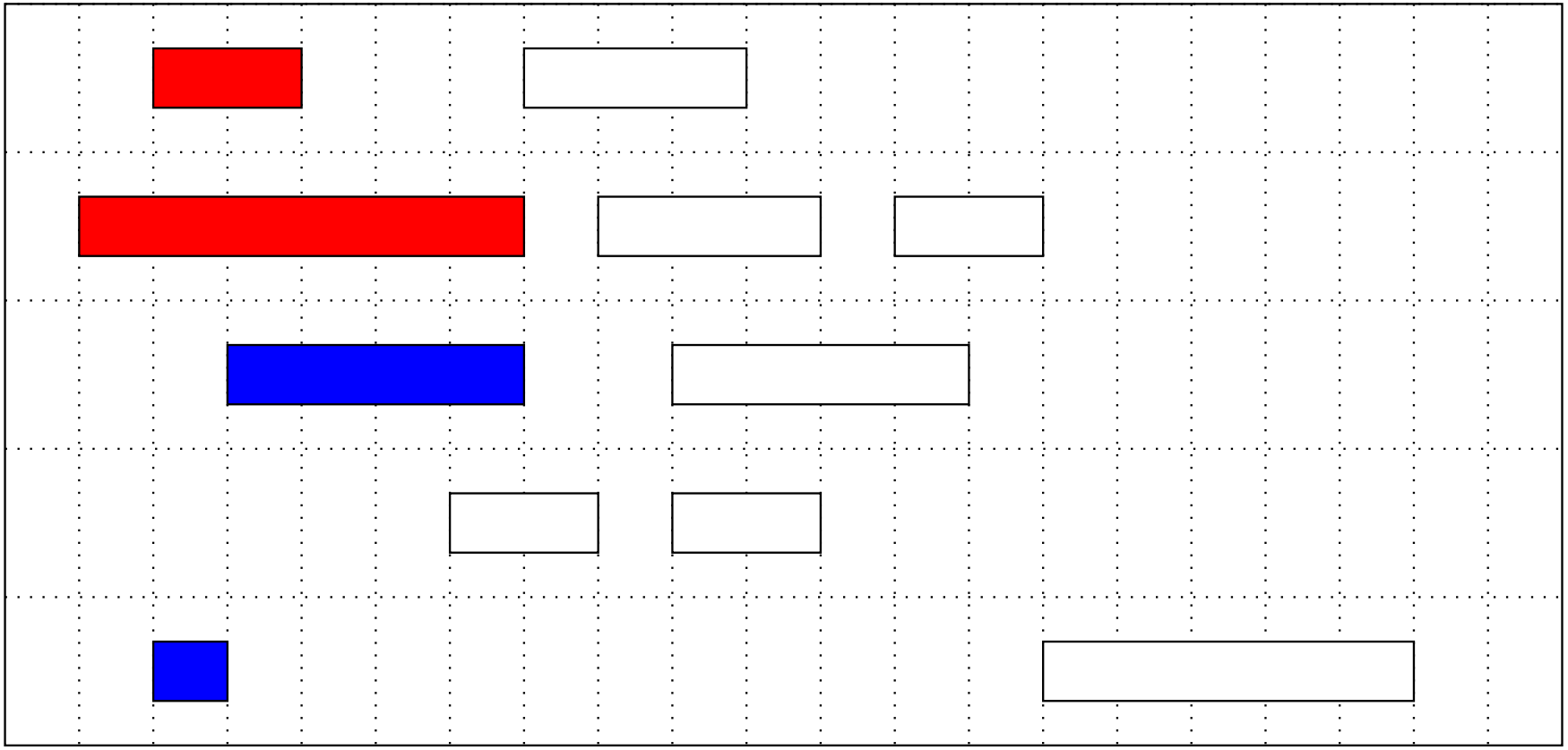
Select request which finishes first

- Remove incompatible requests:



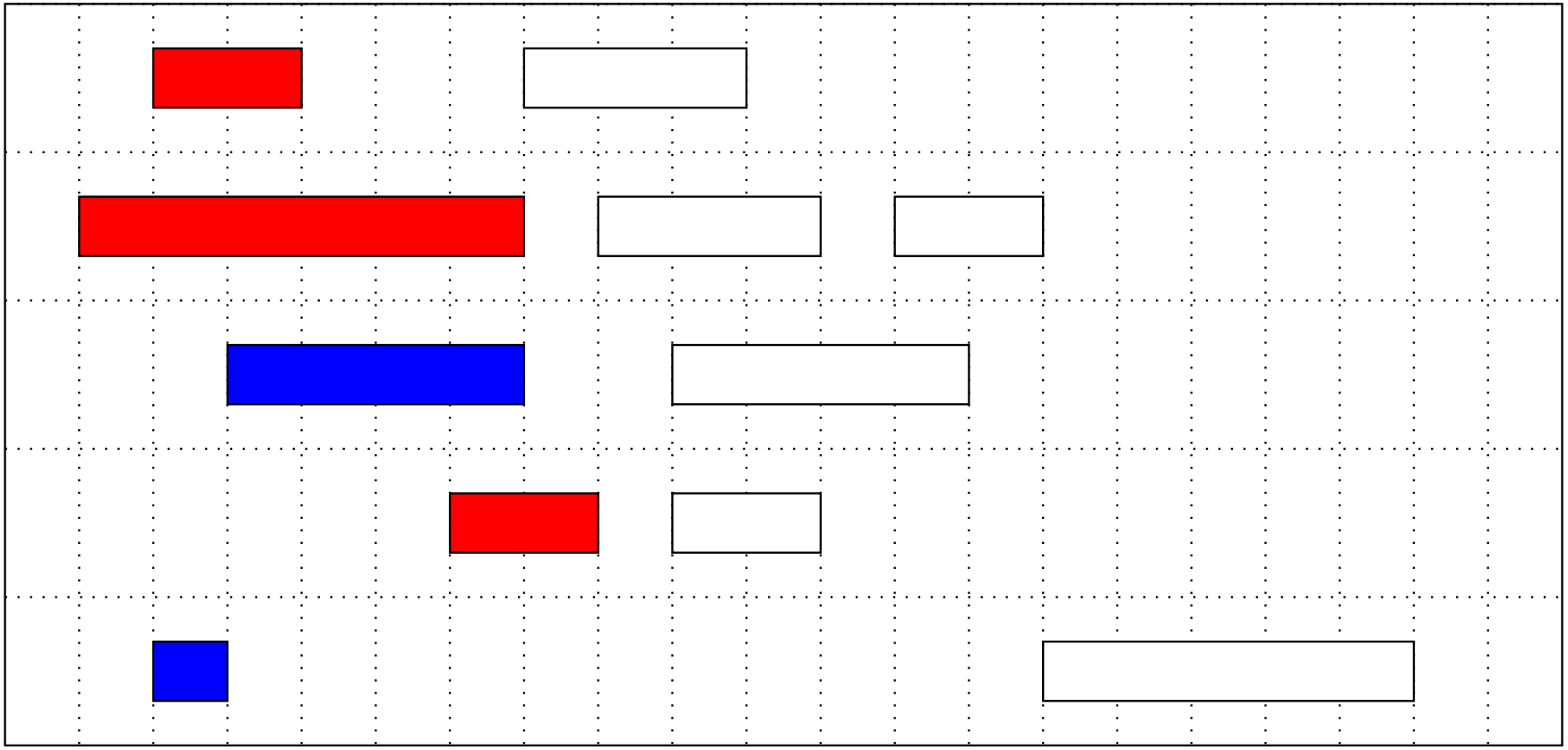
Select request which finishes first

- Select next request:



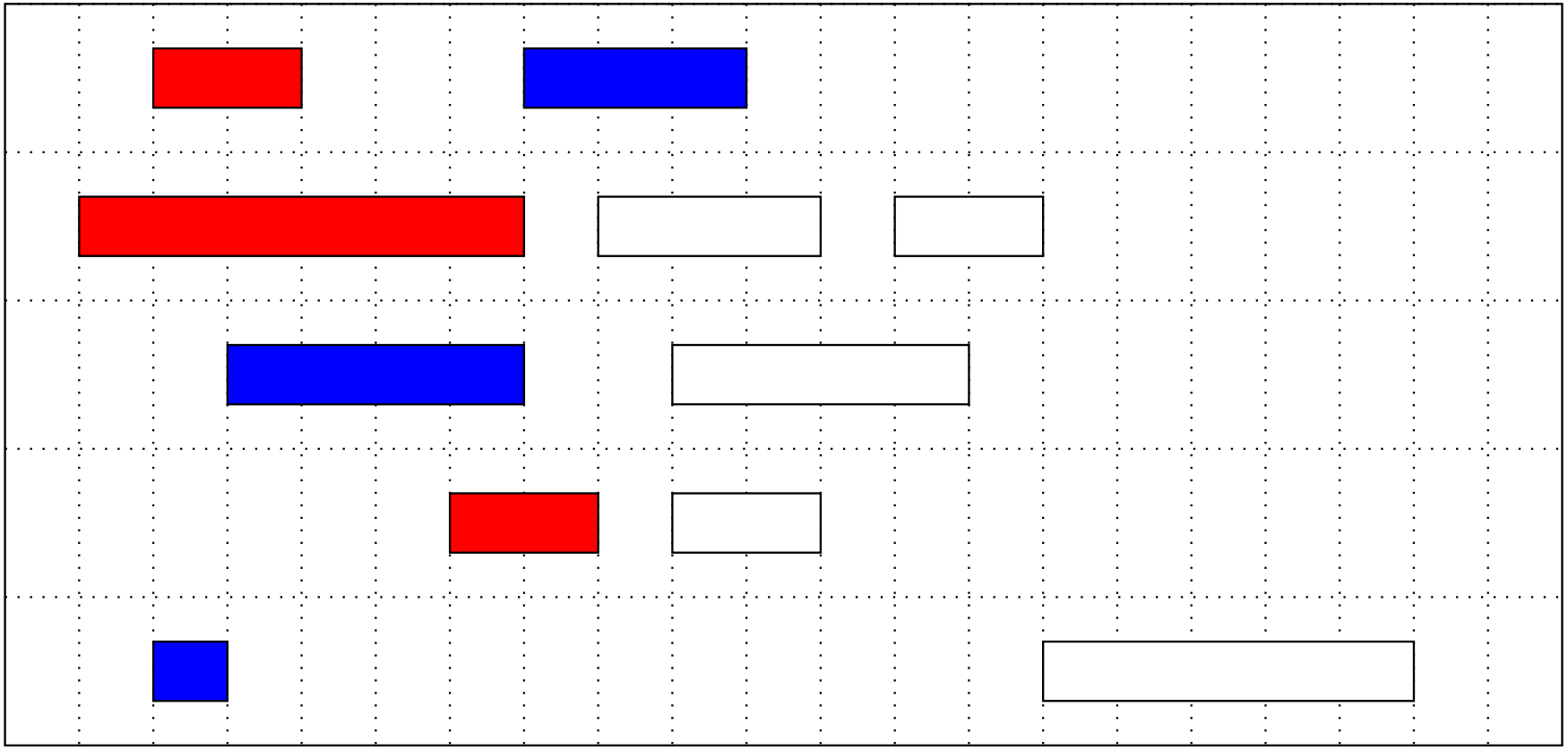
Select request which finishes first

- Remove incompatible request:



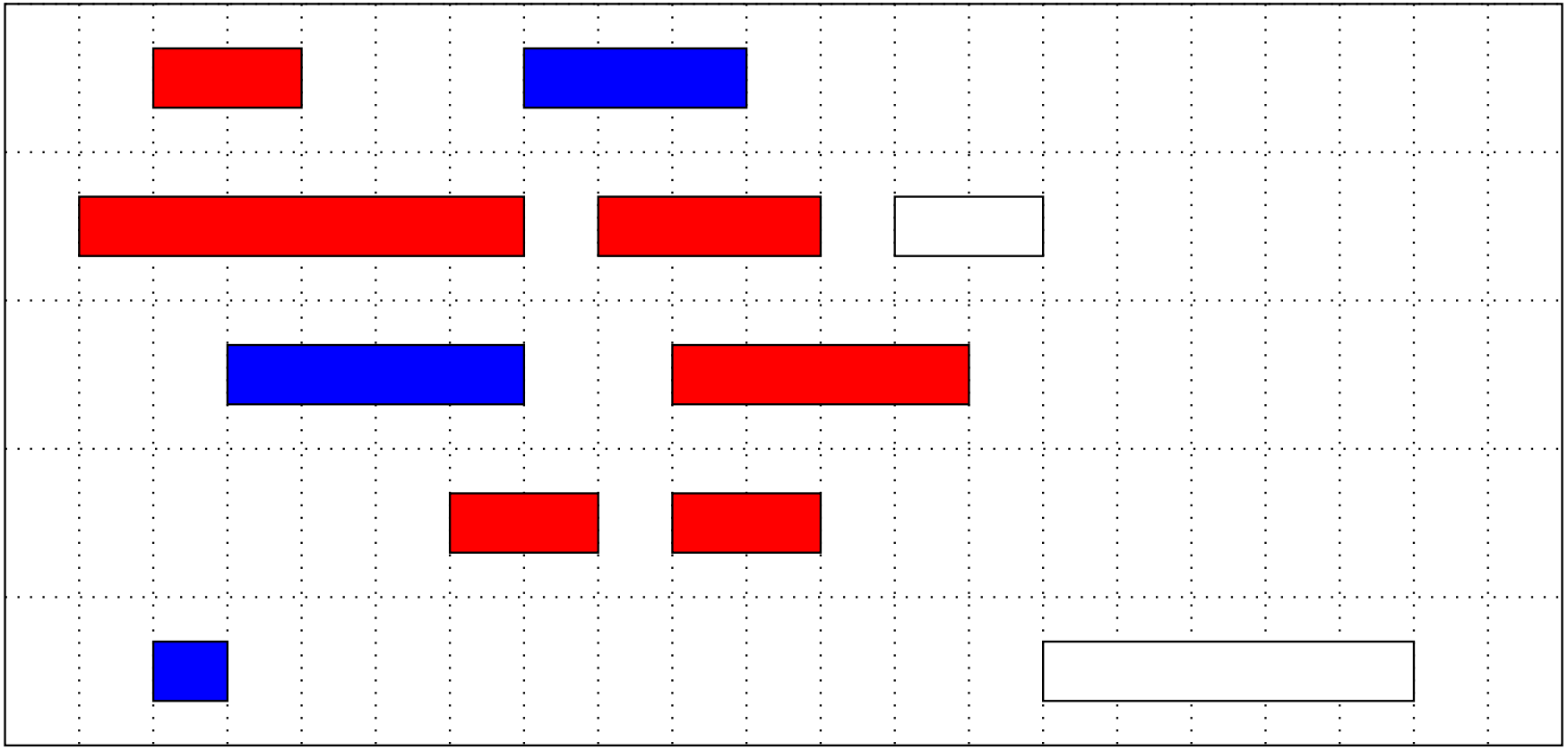
Select request which finishes first

- Select next request:



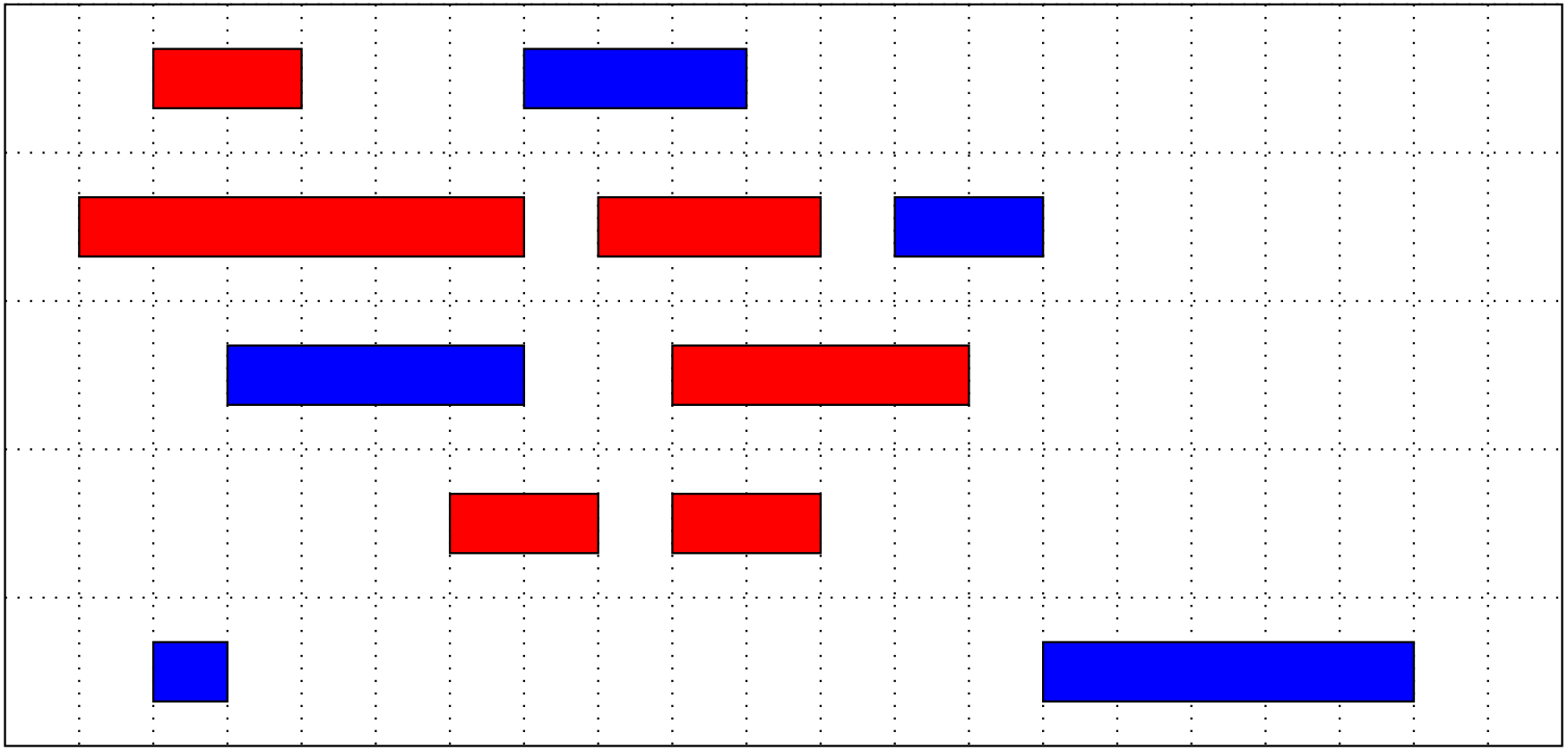
Select request which finishes first

- Remove incompatible requests:



Select request which finishes first

- End result:



Proving optimality of a greedy algorithm

- What should we prove?
- Can there exist several optimal schedules?
- Either show our algorithm is as good as an optimal (stays ahead), or output from an optimal algorithm can be transformed to the output of our algorithm.
- For our problem, assume there is an optimal schedule represented as a sequence T sorted in order of increasing finish time
- We should not try to prove $S = T$
- Instead we should prove $|S| = |T|$
- We will use the first proof technique:
  "our algorithm stays ahead of an optimal solution"
- That is, $|S| \geq |T|$

Two solutions

- Our: $S = (r_1, r_2, ..., r_n)$
- Optimal: $T = (t_1, t_2, ..., t_m)$
- We want to show $n = m$
- S and T are both sorted by increasing finish time
- It is clear that $f(r_1) \leq f(t_1)$ since we select the request with earliest finish time
- There are at least n requests in T so we can aim at proving:
- $f(r_k) \leq f(t_k) \quad 1 \leq k \leq n$

Comparing the solutions

Lemma

$f(r_k) \leq f(t_k) \quad 1 \leq k \leq n$

Proof.

- Proof by induction. $f(r_1) \leq f(t_1)$ is clear.
- For $k > 1$, assume (1): $f(r_{k-1}) \leq f(t_{k-1})$.
- Since T is compatible (2): $f(t_{k-1}) \leq s(t_k)$
- From (1) and (2) follows (3): $f(r_{k-1}) \leq s(t_k)$
- Our algorithm can select t_k as its r_k
- Our algorithm selects as r_k the request with earliest finish time, i.e. $f(r_k) \leq f(t_k)$

$\square$

$|S| = |T|$

- It remains to prove that $|S| = |T|$.
- Recall $n = |S|$ and $m = |T|$.

Theorem

$|S| = |T|$.

Proof.

- Assume in contradiction that $m > n$.
- We know $f(r_n) \leq f(t_n)$
- Since $m > n$, T contains a request t_{n+1}
- We must have $s(t_{n+1}) \geq f(t_n) \geq f(r_n)$
- But this request should have been scheduled by our algorithm which contradicts the assumption that $|S| = n$ so $m = n$ and optimality has been proved

$\square$

Time complexity

- Our greedy algorithm can be implemented in time $O(n \log n)$
- First all requests are sorted in order of increasing finish time
- Then a linear pass finds the schedule

Scheduling to minimize delays

- Again one resource
- Consider now requests with a soft deadline $d(r)$ and a time length $t(r)$
- It is not a disaster to fail a soft deadline compared with a hard deadline
- $s(r)$ and $f(r)$ are start and finish times and in this problem they are output and not input
- The delay of one request is $\max(0, f(r) - d(r))$
- Our problem is to schedule requests so that the maximum delay of any request is minimized
- What is a simple rule to do that optimally?

Algorithm: select next with earliest deadline

- Request are renamed so that $d(r_1) \leq d(r_2) ... \leq d(r_n)$
- Our algorithm simply is to schedule the requests in this order, or in other words, **the earliest deadline first**
- In addition, we schedule requests so that $s(r_{i+1}) = f(r_i)$, i.e. without a gap between r_i and r_{i+1}
- Thus there is no **idle time** between any two requests
- Consider any optimal schedule T. Can it have idle time?
- Yes, e.g. if $t(r_1) = 1, d(r_1) = 2, t(r_2) = 3, d(r_2) = 10$
- With $s(r_1) = 1, f(r_1) = 2, s(r_2) = 7, f(r_2) = 10$, the maximum delay is zero
- But there obviously exists a different optimal schedule without any gap — just start the requests in the same order but as early as possible

Inversions

- Let $d(r_i) < d(r_j)$
- If r_j is scheduled before r_i, it is called an **inversion**
- Our algorithm creates no inversions
- But if $d(r_i) = d(r_{i+1})$ then it can schedule r_{i+1} before r_i

An example

- Let all requests take one time unit
- Let all requests have deadline $d(r_i) = 3$
- $S = (r_1, r_2, r_3, r_4, r_5)$
- $T = (r_4, r_5, r_1, r_2, r_3)$
- In S r_1, r_2, r_3 have no delay, r_4 is delayed 1 and r_5 is delayed 2
- In T r_4, r_5, r_1 have no delay, r_2 is delayed 1 and r_3 is delayed 2
- Same maximum delay but for different requests

First part of our optimality proof

Lemma

All schedules with no idle time and no inversions have the same maximum delay

Proof.

- Consider two different schedules S and T without idle time and no inversions
- Then the only difference between S and T is the order in which requests with identical deadlines are scheduled
- In a sequence with such requests, the last has the maximal delay or no delay.
- The maximal delay is the same in both schedules.

□

Second part

- We will prove that an optimal schedule T can be transformed to S

Lemma

Assume an optimal schedule Q has an inversion of r_i and r_j i.e. $d_i < d_j$ and r_j is scheduled before r_i. Then Q has a pair r_a and r_b of inverted requests scheduled immediately after each other.

- In $(7, 8, 9, 1, 2, 3, 4, 5, 6)$ with $i = 1, j = 7$ we have $a = 1, b = 9$.

Proof.

- We have $(r_j, ..., r_i)$ with k requests scheduled between r_j and r_i, $k \geq 0$
- In this sequence of $k + 2$ requests, since $d_i < d_j$, there must be a first pair r_a and r_b of inverted requests with r_b scheduled immediately before r_a such that $d_a < d_b$.

□

Third part

Lemma

Let r_a and r_b be a pair of inverted requests scheduled consecutively in Q Swapping r_a and r_b into R, the maximum delay does not increase.

Proof.

- We have $Q = (..., r_b, r_a, ...)$ and $R = (..., r_a, r_b, ...)$.
- The delay of r_a cannot have increased in R
- Let $f^X(r)$ be the finish time in schedule X for request r
- Let $t = f^Q(r_a) = f^R(r_b)$ (i.e. finish time of last of them)
- The delay of r_a in Q is $f^Q(r_a) - d(r_a) = t - d(r_a)$
- The delay of r_b in R is $f^R(r_b) - d(r_b) = t - d(r_b)$
- Can r_b now be more late than r_a was? I.e. can $t - d(r_b) > t - d(r_a)$?
- $t - d(r_b) > t - d(r_a) \iff t + d(r_a) > t + d(r_b)$
- No, since by assumption $d(r_a) < d(r_b)$

Last part

Theorem

Our algorithm produces a schedule with minimum maximum delay.

Proof.

- We have just shown that there exists an optimal schedule T with no idle time and no inversions.
- Since all schedules with no idle time and no inversions have the same maximum delay, our algorithm is optimal.

□

An exchange argument

- What did we do?
- We find an algorithm which seems to be optimal.
- We characterize optimal solutions.
- We exchange optimal solutions to produce an output which is identical to ours.
- Therefore our algorithm is optimal.
- Note that the schedules may be different but the maximum delay is the same.